FIT2093 Assignment 1 Tasks 2 & 3 – Semester 1, 2024 Submission Guidelines & Tasks

Guidelines Details

- **Deadline:** Assignment 1 **Task 2 & 3** due date is extended to **Week 10** on **6 May 2024** at **7:55am** Melbourne, Australia time (CL Campus) and **6 May 2024** at **7:55am** Malaysia time (MA Campus). This Task is an individual (not group) work and it must be submitted by each student **individually**.
- **Submission Platform:** Electronic submission via the `Assignment 1 Task 2 & 3 Submission' link on the Moodle Assessments page (an Ed announcement will be sent once the link is added).
- **Required Files:** Required data and numerical parameter files for the assignment will be available for download via the Moodle `Asg 1 Task 2 & 3 Specification Download' link.
- **Submission File Format:** Two PDF documents: one for Task 2 and one for Task 3.
- **Submission Page Limit:** Each submitted PDF document must be **at most 20 pages, excluding cover page and references**. Any screenshots that cannot fit in the main 20 pages can be placed in an Appendix (which does not count in the page limit).
- **Plagiarism:** It is an academic requirement that your submitted work be original. Zero marks will be awarded for the whole submission if there is any evidence of copying, collaboration, pasting from web sites, or copying from textbooks.
- **Use of Generative AI tools:** ChatGPT or other AI tools may be used for study purposes, to learn about your topic, and to develop your assignment. However, similar to citation requirements for other references, you must include a clear declaration of all generative AI tools used (e.g. ChatGPT, DALL-E, Grammarly, voice-to-text), how and where you have used them. Please follow the Monash guidelines on how to acknowledge the use of Generative AI.

Notes

- You can use the `SageMath` tool to perform any calculation necessary for this assignment. The `sagemath` web interface is available at: `https://sagecell.sagemath.org/`. Refer to the Applied Session exercises.

- **For each question**, you need to answer **both** the computation result question **and** the explanation questions about your working process such as the source code or the commands you are using to solve the tasks.

- Note that if numbers in this assignment are specified in hexadecimal format, your written answer and many software packages expect hexadecimal numbers to be input a '0x' prefix (e.g. '0xa0b1c2d3') for indicating the hexadecimal format. For example, this prefix allows SageMath to interpret the value in hex.

Monash University – FIT2093 Assignment 1 Tasks 2 & 3

Marks

• This Assignment 1 Tasks 2 & 3 are weighted **15% each**, and together make up **30%** of the total unit marks.

• The assignment 1 Tasks 2 & 3 are each marked out of **100** nominal marks. Each mark in Task 2 and Task 3 is thus worth 0.15% of the total unit marks.

Task 2 (100 marks)	**Marks**
SubTask 2.1 (30 marks)	● Explanation of decryption steps (10 marks) ● Block diagram of decryption (7 marks) ● OpenSSL commands and decrypted values (7 marks) ● Presentation (6 marks)
SubTask 2.2 (40 marks)	● Discussion of CTR mode security considerations and how well addressed in group encryption (7 marks) ● Explanation of security vulnerability in group encryption (7 marks) ● Explanation of vulnerability exploitation (7 marks) ● Demonstration of attack (7 marks) ● Fix for vulnerability and reasoning (7 marks) ● Visual aids and presentation (5 marks)
SubTask 2.3 (30 marks)	● Explanation of modified group encryption (10 marks) ● Block diagram of modified group encryption (5 marks) ● Explanation of why modification is secure (5 marks) ● Demonstration of modification (5 marks) ● Visual aids and presentation (5 marks)

Task 3 (100 marks)	**Marks**
SubTask 3.1 (40 marks)	● Missing steps for group session init protocol (8 marks) ● Explanation of purpose of missing steps (8 marks) ● Demonstration of missing SESSID and gsk decryption steps with commands (8 marks) ● Correct computation of gsk and SESSID (8 marks) ● Presentation (8 marks)
SubTask 3.2 (40 marks)	● Explanation of both attack scenarios (8 marks) ● Discussion of security of protocols in these scenarios (7 marks) ● Explanation of vulnerability in protocol (5 marks) ● Explanation of attack steps (5 marks) ● Demonstration of attack steps (5 marks) ● Computed attack results (5 marks) ● Visual aids and presentation (5 marks)
SubTask 3.3 (20 marks)	● Explanation and reasoning of modification of protocol (8 marks) ● Block diagram of modification (5 marks) ● Discussion of modification impacts (4 marks) ● Visual aids and presentation (3 marks)

Task 2: Group Message Encryption for *PeerHelper* (15%)

After consideration of the Canary block cipher design, the project manager Reyes decided that due to the more established and higher confidence in the security of AES, the PeerHelper development will proceed with **AES-256** as the block cipher for group message encryption.

For the next stage of the project, Reyes asked you to investigate the design of the **group message end-to-end encryption** method, including using a **block cipher mode** of operation, and evaluating different options for their confidentiality and authentication security.

Reyes proposes that to initialize a group messaging session, group members will run a group session initialization key exchange process, to be investigated later in the following task (Task 3 below). For this group message encryption task, Reyes said you may assume that the group initialization stage has already been executed, resulting in the following information:

- a public 32-bit **session identifier SESSID** known and shared by all group members,
- a private AES-256 **group session key gsk** which is known and shared by all group members,
- a list of private **group member ID**s that belong to the group: **ID_1, ID_2, … ID_n**, where **n** is the number of group members in the session. These group member IDs (96-bit per ID) are known to the group members and the PeerHelper app server.

Reyes sent you the following proposal for the group message encryption method. To send a group message **m** to the group, the sending group member Alice (say Alice is member 1 with identity **ID_1**) does the following:

1. Prepares a 128-bit private **message header hdr** = (SESSID || ID_1).
   Here, || denotes concatenation of strings, e.g. SESSID || ID_1 denotes the concatenation of the two bit strings SESSID (session ID) and ID_1 (sender ID). For example, if SESSID = "9S42" and ID_1 = "AliceKerr000" then hdr = SESSID||ID_1 = "9S42AliceKerr000" (note that the SESSID consists of 4 ASCII characters and the ID_1 consists of 12 ASCII characters, to make the total hdr length equal to 16 ASCII characters x 8 bit/char = 128 bit). Refer to **Fig. 1** below for an illustration.

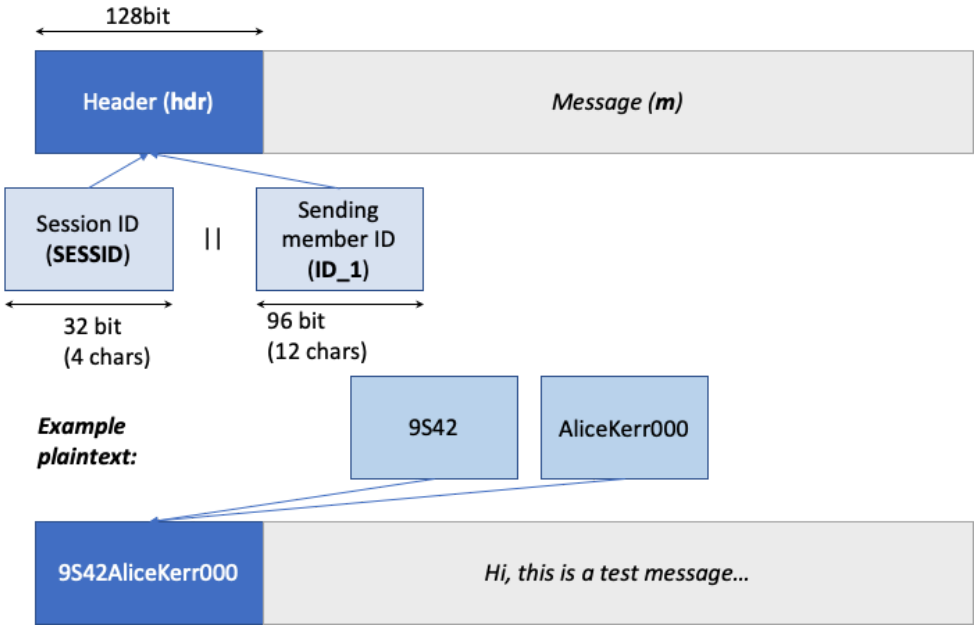


Fig. 1. Illustration of Step 1 of group message encryption.

2. Encrypts the bit string **(hdr || m)** using AES-256 in the CTR mode of operation, with the group session key **gsk** to get a ciphertext **C** (note that **C** has the form IV||C[1]||C[2],...C[N], where IV is the CTR mode Initial Value, and C[1],C[2],...,C[N] are N AES-256 ciphertext blocks, note that in CTR mode, the last ciphertext block sent may be shorter than a full AES block, depending on the length of the plaintext). The Initial Value **IV** for the CTR mode of operation is derived from the time of day **time** (hours:minutes) by hashing: **IV** = H(**time**), where H(**time**) is the leftmost 128-bit of the 256-bit output of the SHA256 cryptographic hash function on input the string **time**. Refer to **Fig. 2** below for an illustration.

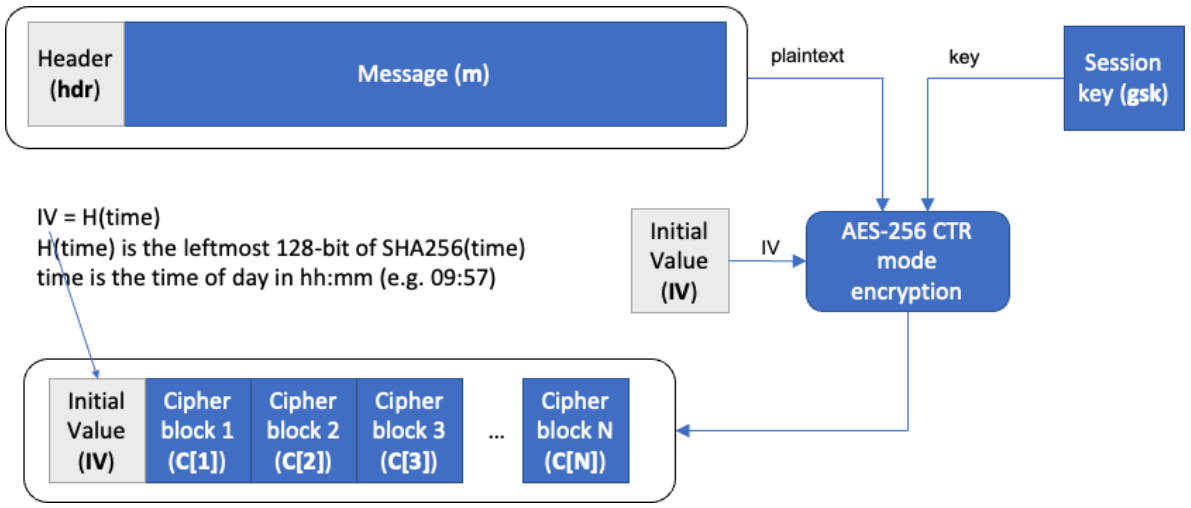


Fig. 2. Illustration of Step 2 of group message encryption.

3. Sends **SESSID || C** to all the members of the group. Refer to **Fig. 3** below.

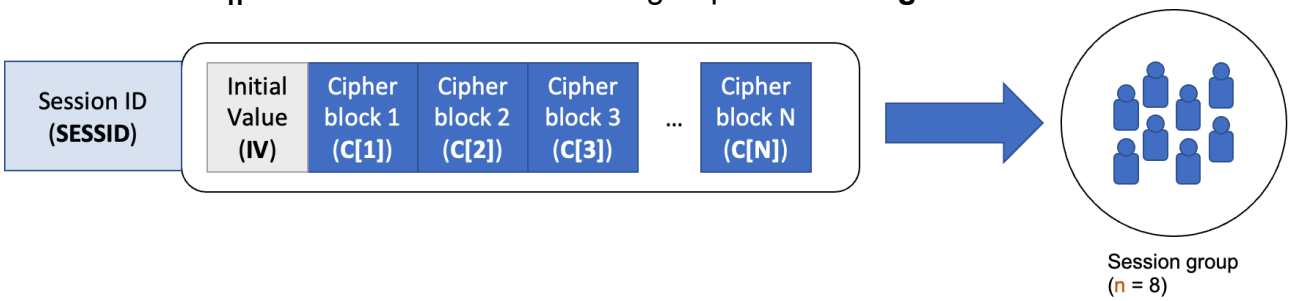


Fig. 3. Illustration of Step 3 of group message encryption.

Reyes asked the developers Kira and Misao to implement this proposal and send you a sample group encryption message SESSID||C encrypted with a sample group session key gsk.

Your goal: Reyes asked you to test and evaluate the functionality and security of this design and implementation of the PeerHelper group message encryption method.

SubTask 2.1: Functionality Testing

2.1 Kira and Misao sent you a sample group encryption message **SESSID||C** encrypted with a sample group session key **gsk,** using the OpenSSL cryptographic tool. The **gsk** and group encryption messages are available in Moodle.

Write a response email to Kira and Misao explaining how you tested decryption of the group encryption message. Include in your response:

- your explanation of the decryption method that you used to test the decryption by each group member's PeerHelper app,
- a block diagram of the decryption method supporting your explanation,
    - in the diagram, indicate the inputs parsed from the given encrypted message together with their corresponding size in bits, and the flow of the block cipher CTR mode operations (including any XOR operations), showing the number of decrypted blocks and what information is in each block.
- your decrypted values of session ID, sender ID, and message obtained from the sample encrypted message using appropriate commands,
- Your reasoning for the number of decrypted blocks used and their sizes, given the length of **m**,
- screenshots of your linux commands for parsing **SESSID||C** into the relevant information and OpenSSL commands in your decryption process. Explain how you obtained each value.

Hint: Refer to Applied Session Week 4-7 for encryption and parsing functions.

SubTask 2.2 Security Evaluation: Confidentiality

To help evaluate the confidentiality of the group message encryption method, Kira and Misao sent you five encrypted group messages **SESSID_1 || C_1, SESSID_2 || C_2, SESSID_3 || C_3, SESSID_4 || C_4, SESSID_5 || C_5** sent by group members during the indicated times (see Moodle).

***Hint*:** To read the cipher binary file, use "`bless <filename>`" or "`xxd <filename>`" to copy the relevant part of hex values.

Your task: Examine the group encryption method in **Figs**. **1-3**. Based on the discussion in this unit, consider:

- how block cipher modes of operation should be used securely
- common insecure misuses of them

Consider whether you think the group encryption method from **Figs**. **1-3** is secure or insecure in terms of **confidentiality**, along with the reason.

Write an email to explain your confidentiality security findings to Kira and Misao. Your email should include:

- discussion of the important security considerations for the CTR mode of operation and how well you think they are addressed in this group encryption method in **Figs**. **1-3**,
- explanation of any security vulnerability you identified in the encryption method,
- explanation of how one such vulnerability could be exploited by an attacker **Zoe**, who eavesdropped on the encrypted group messages, to reveal some private information (such as private identities of group members who sent the intercepted messages or the message contents),
- what private information can be revealed by Zoe, from the given encrypted messages and the message sending time,
- a recommendation to Kira and Misao on how to fix the vulnerability, and the reasons why it fixes the problem,
- screenshots of any OpenSSL and/or SageMath commands you used in the decryption and your explanations of how you got each value. You can acquire the values from the given message by using the parsing techniques in Applied Session Week 7.

Hint*:** You may assume that prior to her attack, **Zoe** found out the ID "**Delta Zhang00**" who sent the third message intercepted by **Zoe. Zoe** found this out by overhearing "**DeltaZhang00**"s private conversation. However, prior to her attack, **Zoe** did ***not find the IDs of any other group members nor the group session key **gsk**.

SubTask 2.3 Security Evaluation: Integrity/Authenticity

Kira and Misao came back to you for help on integrity evaluation of the group encryption method. They realised that some group insiders (i.e. group members) may try to attack the integrity of the system, and such attacks should also be prevented. Based on your studies in this unit, think about how the group encryption method could be modified to protect against ***group insider*** integrity attacks.

Your task: Write a follow-up email to Kira and Misao to describe your modified group encryption method. Your email should include:

- an explanation of your modified group encryption method,
- a block diagram to illustrate your method,
- an explanation of why your method would protect against attacks by a malicious group insider "BobHowes0000" who *intercepts and modifies* a group encryption message sent by honest group member "DeltaZhang00" to a new encrypted message delivered to all other group members,
- explain in particular why, with your encryption method, it would not be feasible for "BobHowes000" to modify the group encryption message sent by "DeltaZhang00" such that all group members decrypt **a different** message but are fooled to think that the message was sent by "**DeltaZhang00**",
- explain any other assumptions (e.g. on the group key distribution setup phase) that your fix entails.
- an example encrypted group message that would be computed by "DeltaZhang00" with your modified group encryption method. For this example, you may modify the encrypted group message **SESSID_3 || C_3** sent by "DeltaZhang00" in SubTask 2.2,
- screenshots of any commands used and any other additional values generated by Delta to compute your example modified encrypted group message.

***Hint*:** Your modified encrypted message should have the form "SESSID_1|| C_1||Y", where Y is some additional value.

Submission: Submit your Task 2 report as a PDF file at the Moodle Assessment Page 'Task 2&3 submission link.

Task 3: User Enrolment and Group Session Initialization for *PeerHelper* (15%)

To initialize a group messaging session, group members will run a group session initialization **key exchange** process, so that the resulting session key can be used for the group session encryption protocol from Task 2. Your task now is to help the ***PeerHelper*** developers Kira and Misao to understand, analyse and improve the security of the group session initialization key exchange protocol proposed by the manager Reyes. Reyes has provided his proposed **two** protocols which are:

(1) A **user enrolment protocol** illustrated in **Fig. 4** and detailed in **Fig. 6**.

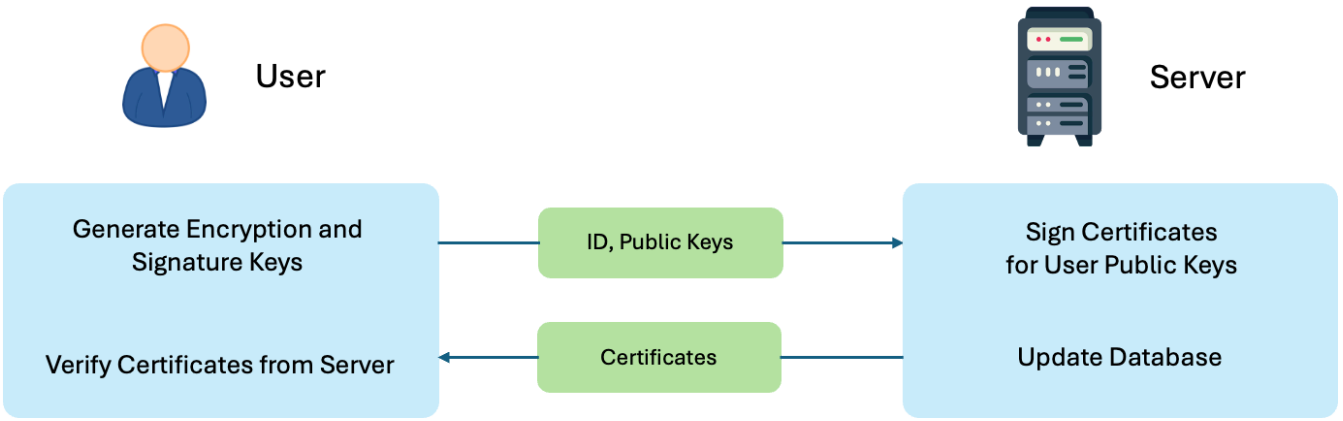


Fig. 4. Illustration of the PeerHelper **user enrolment protocol**.

(2) A **group session initialization protocol** run by the group session Initiator User, illustrated in **Fig. 5** and detailed in **Fig. 7.**

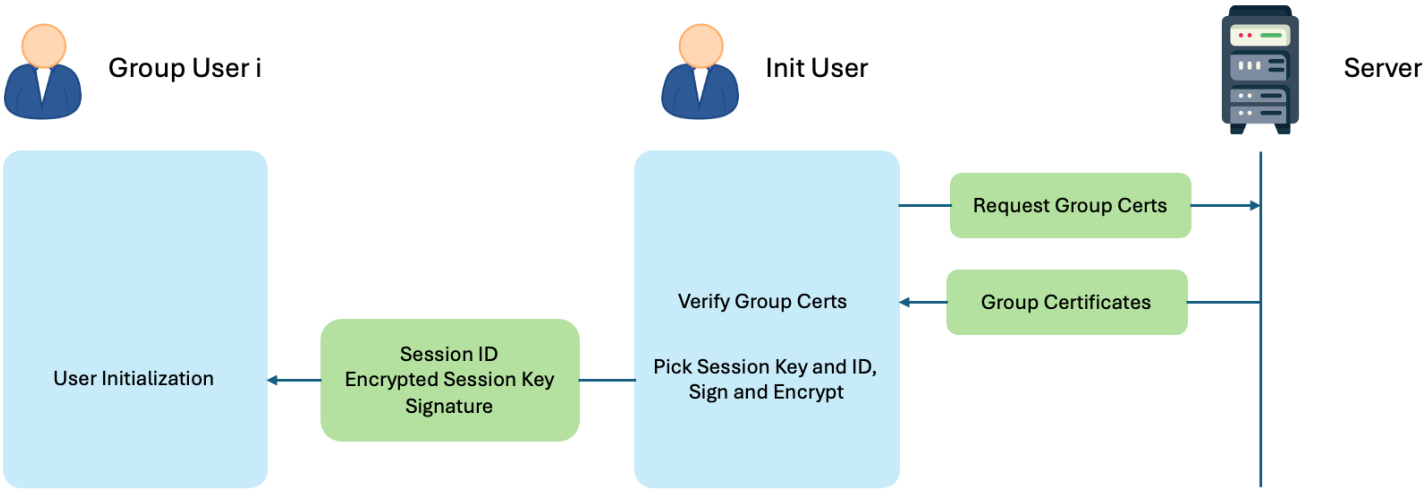


Fig. 5. Illustration of the PeerHelper **group session initialization protocol**.

SubTask 3.1: Completing the Group Session Initialization Protocol

In order for Kira and Misao to complete their implementation of the proposed protocol, they need to also know the protocol steps to be followed by the other group members (besides the group session Initiator User) in the protocol.

Write an email to Kira and Misao to explain the steps to be performed by the group member ID_i (for any i=2,...,n) upon receiving the group session initialization message from the group initiator. Your email should:

(1) detail the missing steps at the end of **Fig. 7**,
(2) explain the purpose of each missing step,
(3) demonstrate an example of how to carry out the **SESSID extraction and group session key gsk decryption steps** on the group session protocol message given on Moodle (note: you do ***not*** need to demonstrate the other missing steps from (1) and (2) on the message given on Moodle).

For your example **(3)** above of the SESSID extraction and gsk decryption step in the group session initialization protocol, assuming you are one of the group users to join a session initiated by Initiator, you are given on Moodle:

- two key-pairs generated in user enrolment protocol,
- a message from Initiator, (**SESSInit**, Cert_{1,s}, SESSID, C_i, sig_i) in step 6 of **Fig. 7**,
- the server's signature public key, PK_{S,s} for verification of signature.

In part **(3)** above of the email, you should

- demonstrate how to obtain the group session key **gsk**,
- show your computed values of the SESSID and **gsk**,
- provide screenshots of GPG commands to show workings and intermediate values to explain your output results. In part (3) of email, you **only need to demonstrate with screenshots the commands for extraction of SESSID and group session key GSK, no other GPG commands need to be demonstrated**.

Hints:

- To read the group session key file, use the command "`xxd -ps <output file>`"
- Use suitable variants of the GPG commands you have learnt in the Week 5 Applied Session and relevant commands in the Week 7 Applied Session.

Reyes' proposed a user enrolment protocol run between a user with identifier ID enrolling in the PeerHelper App and the PeerHelper App server is shown in **Fig. 6.**

User input: ID, PK_{S,s}where:

- ID is the 12-character (96 bit) user ID string used to enrol in the PeerHelper app
  - Note: the ID is entered by the user when installing the PeerHelper App
- PK_{S,s} is the PeerHelper server's signature public key
  - Note: the PeerHelper Server's public key PK_{S,s} is hard-coded into the PeerHelper App
  - Note: In PK_{S,s}, S denotes Server, s denotes signature

Server input: (SK_s, DBID), where:

- SK_{S,s} is the PeerHelper server's signature private key
- DBID is the database of registered user IDs and Certs

1. **User generates RSA signature & encryption keys**:
   a. User generates an RSA 2048-bit signature key pair (SK_s, PK_s).
   b. User generates an RSA 2048-bit encryption key pair (SK_e, PK_e).

2. **User sends ID & public keys to Server:**
   a. User sends a message (**Enrol**, ID, PK_s, PK_e) to the Server.

3. **Server signs certs for user public keys:** Upon receiving (**Enrol**, ID, PK_s, PK_e), Server does the following:
   a. **If** ID or PK_s or PK_e is in the database DBID of enrolled users:
      i. Server sends an error message (**Error**, "Error: already enrolled") to User.
   b. **Else**,
      i. Compute signed certificates for user ID public keys:
         1. Server computes a signature sig_{S,s} on (ID, PK_s) using SK_{S,s}.
         2. Server sets Cert_s = (ID, PK_s, sig_{S,s})
         3. Server computes a signature sig_{S,e} on (ID, PK_e) using SK_{S,s}.
         4. Server sets Cert_e = (ID, PK_e, sig_{S,e})
      ii. Server sends enrolment confirmation message (**Enrolled**, ID, Cert_s, Cert_e) to User.
      iii. Server adds (ID, Cert_s, Cert_e) to user database DBID.

4. **User verifies certs from Server:** Upon receiving message (**Enrolled**, ID', Cert'_s, Cert'_e), where Cert'_s = (ID'_s, PK'_s, sig'_s) and Cert'_e = (ID'_e, PK'_e, sig'_e)), User does the following:
   a. Verify that ID'_s = ID'_e = ID and PK'_s = PK_s and PK'_e = PK_e.
   b. Verify using PK_{S,s} that sig'_{s} is a valid signature by Server on Cert'_s and sig'_{e} is a valid signature by Server on Cert'_e.
   c. If any verification fails,
      i. re-start user enrolment protocol

User Output: User's certificates Cert'_s and Cert'_e stored in User's App.
Server Output: Updated user database DBID with enrolled user ID record.

Fig. 6. Specification of the PeerHelper **user enrolment protocol**.

The **group session initialization protocol** is run between an initiator user ID_1 that initiates a group session and a group of *n* users (ID_1,...,ID_n) and is shown in **Fig. 7.**

Initiator User input: ID_1, SK_{ID_1, s}, Cert_{ID_1,s}, Cert_{ID_1,e}, PK_{S,s}, ID_2, … , ID_n where:

- ID_1 is the 12-character (96 bit) user ID string of Initiator User.
- SK_{1,s} is the signature private key of Initiator User.
- Cert_{1,s} and Cert_{1,e} are the certificates of Initiator User ID_1
- PK_{S,s} is the PeerHelper server's signature public key
  - Note: PeerHelper Server's public key PK_{S,s} is hard-coded into the PeerHelper App
- ID_2, … , ID_n are the other n-1 group member user ids (besides the Initiator User).

Server input: DBID, where:

- DBID is the database of registered user IDs and Certs.

Group User i input (i=2,...n): , where:

- DBID is the database of registered user IDs.
- SK_{ID_i,e}is the encryption private key of User i.

1. **Initiator User requests group certs from Server:** Initiator User sends a message (**ReqCerts**, ID_1, ID_2, … , ID_n) to Server.

2. **Server sends group certs to Initiator User:** Upon receiving message (**ReqCerts**, ID_1, … , ID_n), Server retrieves CertList = (Cert_{2,s}, Cert_{2,e}), … , (Cert_{n,s}, Cert_{n,e}) for users ID_2, … , ID_n (respectively) from DBID and sends the message (**CertResp**, CertList) to Initiator User ID_1.

3. **Initiator User verifies group certs:** Upon receiving Server response message (**CertResp**, CertList'), where
   a. CertList' = (Cert'_{2,s},Cert'_{2,e}), (Cert'_{3,s}, Cert'_{3,e}), … , (Cert'_{n,s},Cert'_{n,e})
   b. And for i = 2, … ,n
      ii. Cert'_{i,s} = (ID'_{i,s}, PK'_{i,s}, sig'_{i,s})
      iii. Cert'_{i,e} = (ID'_{i,e}, PK'_{i,e}, sig'_{i,e})

   Initiator User does the following:

   c. If ID'_{i,e} or ID'_{i,s} are not equal to ID_i
      iv. Initiator User restarts the protocol.

4. **Initiator User picks group session key and session ID:** Initiator User chooses a random AES-256 group session key gsk and a random 4-digit session id string SESSID.

5. **Initiator User encrypts & signs group session key to group**: For i = 2 … n, Initiator User sends message (**SESSInit**, Cert_{1,s}, SESSID, C_i, sig_i) to User ID_i, where:
   - Initiator User computes C_i as the public-key ciphertext produced by encrypting gsk to the receiver key PK'_{i,e}.
   - Initiator User computes sig_i as the signature on message (**SESSInit**, Cert_{1,s}, SESSID, C_i), using the Initiator User's private key SK_{1,s}.

6. **Group User i initialization:** For i = 2 … n, upon receiving message (**SESSInit**, Cert'_{1,s}, SESSID', C'_i, sig'_ i), Group User i does the following:
   - …[To be Completed by Students in SubTask 3.1]

Group User 1 … n Output: session ID SESSID, group session key gsk.

Fig. 7. Specification of the **PeerHelper group session initialization protocol**.

SubTask 3.2: Confidentiality of the Group Session Initialization Protocol

Now that the **group session initialization protocol** description is complete, Reyes asked you to help with evaluating its security. First, consider attacks against the protocol by a group session outsider (who may be a user enrolled in the PeerHelper App but not one of the group session members) that conducts a **passive attack (i.e. eavesdropping)** on the protocol messages and tries to break the confidentiality of the group session. Then, consider attacks by a group session outsider that conducts an **active attack** modifying the protocol messages and tries to break the confidentiality of the group session.

Write an email to Reyes to:

- explain the above potential threats and information available to the attacker in the two (passive and active) attack scenarios,
- Discuss whether the protocols proposed in **Fig. 6** and **Fig. 7** can achieve confidentiality for the group session in each of these scenarios.
- explain your reasoning and evaluation and your assumptions,
- in the case where you think the protocol is insecure in an attack scenario,
    - explain the steps an attacker could take to break the confidentiality of **the group session key gsk**,
    - demonstrate an example attack using GPG commands using the example protocol instance provided to you in Moodle.
    - be sure to include screenshots of your attack commands and intermediate and output results.
    - explain how the protocol in **Fig. 6** and/or **Fig. 7** could be modified to prevent this attack, explaining your reasoning and any assumptions you make.

For your demonstration attack, you may use the following:

- assume the Initiator User, “DeltaZhang00”, requested a CertList from the Server. An attacker can capture all the protocol messages including the CertList received by “DeltaZhang00” in **Step 3** of **Fig. 7**. The CertList is provided on Moodle.

- a program PeerHelper provided in Moodle is running at the Initiator User “DeltaZhang00”, to generate the protocol message **(SESSInit, Cert_{1,s}, SESSID, C_i, sig_i)** sent by the Initiator User in **Step 5** of **Fig. 7**, from the input **(CertList, ID_2, ID_3, … ID_n)** given to the Initiator by the Server in **Step 3** of **Fig. 7**.

Instructions for running the PeerHelper program for demonstrating an attack:

1. Download **`PeerHelper_x86`** or **`PeerHelper_M1`** from Moodle in your VM
2. Run command in VM **`chmod +x PeerHelper_x86`** (or **`PeerHelper_M1`** for M1/M2)
3. Download & store key pairs of Initiator, **`DeltaZhang00_s.key`** from Moodle in your VM
4. Run command **`gpg --import`** to import the Initiator User’s key pairs
5. Download CertRespFile, **`CertResp-<ID_2>-<ID_3>-<ID_4>.json`**, into the same folder as PeerHelper
6. Run command **`./PeerHelper_x86`** (or **`PeerHelper_M1`** for M1/M2) **`--peer <ID_2> <ID_3> <ID_4> --certresp <CertRespFile>`**
7. Three corresponding output files containing **`(SESSInit, Cert_{1,s}, SESSID, C_i, sig_i)`** for each **<ID_2>, <ID_3> & <ID_4>** can be seen in the same folder.

Hint: To convert the output of a gpg into json format, use the following command:

```
gpg [relevant gpg commands output in PEM format using -armour]| awk '{printf "%s\\n", $0}
```

SubTask 3.3: Post-Quantum Security for PeerHelper

Reyes sends you an email saying that he is worried about attackers storing group session initiation ciphertexts and decrypting them in a few years once quantum computers become sufficiently developed to break the 2048-bit RSA encryption used in the PeerHelper **group session initialization** protocol.

On the other hand, Reyes does not want the confidentiality of the system to rely solely on the security of quantum-safe cryptography, since it is very new and its existing implementations may have some vulnerabilities. Therefore, Reyes asks you to **propose a modification** to the PeerHelper **group session initialization** protocol, such that the confidentiality of today's session messages will be maintained into the future as long as either RSA or a quantum-safe encryption algorithm are secure. He would also like you to evaluate the computation and communication costs of such a modification.

Write a brief report for Reyes to:

- explain how you think the PeerHelper **group session initialization protocol** in **Fig. 7** could be modified to achieve the desired security goal outlined by Reyes.
- include a block diagram to illustrate your modification to the protocol,
- explain the reasoning behind your modified steps,
- discuss the impact of the modifications on the communication costs of the protocol.

Hint: Refer to the discussion in Week 7 on Post-Quantum Cryptography.

Submission: Submit your Task 3 report as a PDF file at the Moodle Assessment Page Task 2 & 3 submission link.

Asg 1 Tasks 2&3 Spec Doc Changes Log

- **Released (v1):** 12 April 2024
- **v1.1 : Typo "Fig. 3" → "Fig. 7" fixed on p.9 and p.13**: 18 Apr 2024.
- **v1.2 : Clarifications/Corrections:** 19 Apr 2024
  - **p. 4**: Changed notation in Task 2 for number of group session members to **n** (from previous N). This is for consistency with the notation n used in Task 3, and to avoid confusion with the notation for the number of ciphertext blocks N in Task 2; note that the number of session group members n may not be equal to number of ciphertext blocks N. Accordingly, also changed notation for the number of group session members N –> **n** in Fig. 3 on p. 5.
  - **p. 5**: added clarification "**note that in CTR mode, the last ciphertext block sent may be shorter than a full AES block, depending on the length of the plaintext**."
  - **p. 6**: clarified the dot point "Confirmation that the number of decryption blocks matches what you would expect, given the length of m" → "**Your reasoning for the number of decrypted blocks used and their sizes**, given the length of m."
- **v1.3 : Clarifications/Corrections:** 26 Apr 2024
  - **p. 9:** added clarification on SubTask 3.1: "In part (3) of email, you **only need to demonstrate with screenshots the commands for extraction of SESSID and group session key GSK, no other GPG commands need to be demonstrated**".
  - **p. 12:** added a hint on SubTask 3.2: "**Hint: To convert the output of a gpg into json format, use the following command:**
    ```
    gpg [relevant gpg commands output in PEM format using -armour]| awk '{printf "%s\\n", $0"
    ```
  - **p. 12:** revise **`PeerHelper-32`** to `PeerHelper_x86` (or **`PeerHelper_M1`** for M1/M2)
- **v1.4** : **Due date extended:** 26 Apr 2024
  - **p.1:** `Due date is extended to 6 May 2024.`
- **v1.5**: **Corrections:** 29 April 2024
  p.7: "such that all group members decrypt **a different** message but are fooled to think that the message was sent by "**DeltaZhang00**"